

Product Requirement Document (PRD)

IoT Smart Frame Ecosystem (Backend & API Bridge)

Versi: 1.0 (Final Spec)

Target Agent: Full-Stack AI Developer / Node.js
Agent

Tanggal: Juni 2026

1. Ringkasan Eksekutif & Tujuan

Dokumen ini mendefinisikan kebutuhan teknis untuk membangun sistem **Backend & API Bridge** berbasis Node.js (Express.js) untuk mengelola perangkat IoT berupa *Smart Frame / Pigura Elektronik*. Perangkat keras menggunakan mikrokontroler ESP32 dengan layar 3-Color E-Paper (Hitam, Putih, Merah) beresolusi 250x122 piksel.

Sistem ini berfungsi sebagai jembatan komunikasi jarak jauh (LDR). Pengguna dapat mengirimkan pesan teks romantis atau gambar/lukisan secara asinkron melalui integrasi otomatisasi (WhatsApp Bot Framework), lalu menyimpannya ke dalam antrian server. Perangkat keras ESP32 akan mengambil data tersebut secara efisien guna mendukung mode hemat daya baterai (Deep Sleep) maupun mode instan kabel.

Catatan untuk AI Agent: Implementasikan seluruh endpoint, manajemen status, dan skrip pemrosesan citra digital di bawah ini menggunakan Node.js, Express, dan pustaka manipulasi gambar (seperti Sharp atau Jimp) secara modular.

2. Arsitektur Perangkat & Alur Data

Sistem terbagi menjadi tiga entitas utama yang saling berkomunikasi melalui protokol HTTP REST:

- Sisi Client/Pengirim (WhatsApp Bot Automation):** Mengirimkan instruksi teks atau berkas gambar mentah (JPEG/PNG) ke Backend menggunakan API Key khusus.
- Sisi Server (Node.js & Express Backend):** Menerima data, memproses konversi warna gambar mentah menjadi matriks biner/Hex khusus E-Paper, dan mengelola antrian pesan (Queue Status).
- Sisi Device (ESP32 IoT Frame):** Melakukan *polling* berkala (saat mode baterai) atau mendengarkan data (saat mode kabel) untuk mengambil payload teks atau matriks gambar terbaru.

3. Kebutuhan Fungsional Backend

3.1 Keamanan & Otentikasi

- Otentikasi berbasis token sederhana menggunakan header `X-API-KEY` untuk semua rute internal.
- Mencegah akses ilegal dari pihak luar yang mencoba menembak endpoint perangkat atau server.

3.2 Engine Pemrosesan Gambar (3-Color Image Processor)

Ini merupakan fitur krusial. Ketika menerima berkas gambar dari WhatsApp Bot, server wajib mengolah citra dengan spesifikasi sebagai berikut:

- **Resize & Crop:** Ubah paksa dimensi gambar menjadi tepat **250 x 122 piksel** (sesuai resolusi fisik layar E-Paper 2.13 inci) dengan orientasi lanskap.
- **Kuantisasi 3 Warna:** Ekstrak komponen warna menjadi 3 nilai absolut: Hitam (0), Putih (1), dan Merah (2) berdasarkan batas ambang (thresholding) warna RGB tertentu.
- **Dithering (Opsional - Sangat Direkomendasikan):** Gunakan algoritma Floyd-Steinberg dithering agar gradasi foto/lukisan wajah terlihat halus seperti sketsa koran.
- **Output Hex Array:** Generate dua buah array biner terpisah dalam format Hex Array (Buffer / C++ style array):
 1. `buffer_black` : Bit bernilai 0 untuk piksel hitam, 1 untuk putih.
 2. `buffer_red` : Bit bernilai 1 untuk piksel merah, 0 untuk warna selain merah.

3.3 Manajemen Antrean (Message Queue Logic)

- Mendukung penyimpanan pesan dengan status `PENDING` (belum diambil oleh ESP32) dan `DELIVERED` (sudah sukses ditampilkan di perangkat).
- Jika ada pesan baru masuk sementara pesan lama masih berstatus `PENDING`, mutasikan pesan lama menjadi `SUPERSEDED` atau timpa langsung agar perangkat selalu mendapatkan data paling mutakhir.

4. Spesifikasi API (API Contracts)

AI Agent wajib membuat routing Express.js yang mematuhi kontrak skema data berikut:

4.1 Kirim Pesan Teks (Dari WA Bot / Eksternal)

Karakteristik	Detail
Endpoint / Method	POST /api/v1/messages/text
Header	X-API-KEY: secret_token_key, Content-Type: application/json
Payload Request	<pre>{ "text": "Selamat ulang tahun sayang! Semangat nugasnya hari ini ya <3", "sender": "Wahyu" }</pre>
Response (201)	<pre>{ "success": true, "message_id": 101, "status": "queued" }</pre>

4.2 Kirim Gambar/Lukisan (Dari WA Bot / Eksternal)

Karakteristik	Detail
Endpoint / Method	POST /api/v1/messages/image
Header	X-API-KEY: secret_token_key, Content-Type: multipart/form-data
Body Parameters	image : File binary (PNG/JPEG)
Response (201)	<pre>{ "success": true, "message_id": 102, "status": "processed", "info": "Image successfully converted to 250x122 BWR Hex arrays" }</pre>

4.3 Polling Data Perangkat (Oleh Perangkat ESP32)

Endpoint ini diakses oleh ESP32 ketika baru bangun dari Deep Sleep atau dijalankan secara real-time via kabel.

Karakteristik	Detail
Endpoint / Method	GET /api/v1/device/poll
Header	X-API-KEY: device_secret_token
Response (200 OK) - Jika Teks	<pre>{ "has_update": true, "message_id": 101, "type": "text", "payload": { "text": "Selamat ulang tahun sayang! Semangat nugasnya hari ini ya <3", "timestamp": "2026-06-08T15:30:00Z" } }</pre>
Response (200 OK) - Jika Gambar	<pre>{ "has_update": true, "message_id": 102, "type": "image", "payload": { "width": 250, "height": 122, "hex_black": [0xFF, 0x00, 0xAA, "...", 0x12], "hex_red": [0x00, 0xFF, 0x55, "...", 0xEE], "timestamp": "2026-06-08T15:32:00Z" } }</pre>
Response (200 OK) - No Update	{ "has_update": false }

4.4 Update Telemetri & Status Perangkat

Karakteristik	Detail
Endpoint / Method	POST /api/v1/device/status
Payload Request	<pre>{ "battery_voltage": 4.12, "battery_percentage": 95, "power_mode": "battery", "rssi": -65 }</pre>

5. Desain Skema Database (Minimalist SQLite / PostgreSQL)

AI Agent disarankan mengimplementasikan skema database relasional minimalis menggunakan ORM seperti Sequelize atau Prisma:

Tabel: `messages`

Nama Kolom	Type Data	Atribut / Deskripsi
<code>id</code>	Integer	Primary Key, Auto Increment
<code>type</code>	String / Enum	'text' atau 'image'
<code>text_content</code>	Text	Nullable (Isi ucapan jika bertipe teks)
<code>hex_black_array</code>	Text / BLOB	Nullable (Data string hex array hitam untuk E-paper)
<code>hex_red_array</code>	Text / BLOB	Nullable (Data string hex array merah untuk E-paper)
<code>status</code>	String / Enum	'pending', 'delivered', 'superseded'
<code>created_at</code>	Timestamp	Waktu pembuatan data

6. Non-Functional & Petunjuk Tambahan AI Agent

- **Eksekusi Cepat:** Proses manipulasi citra digital pada rute unggah gambar tidak boleh memblokir thread utama Node.js. Gunakan `async/await` atau `worker threads` jika diperlukan.
- **Dockerization:** Sediakan `Dockerfile` dan `docker-compose.yml` agar aplikasi backend ini siap di-deploy secara instan di Virtual Private Server (VPS).

- **Log & Monitor:** Berikan penanganan error (error handling middleware) yang jelas serta log info konsol menggunakan `morgan` atau `winston` untuk mempermudah pelacakan pengiriman paket dari ESP32.